SCRUM-12 — Plan de tests et scénarios d'acceptation

Les tests logiciels constituent une étape essentielle de la validation de la qualité d'un système. Selon le Cours 10, ils regroupent les activités permettant de vérifier et de valider qu'un logiciel fonctionne conformément aux exigences, répond aux besoins des utilisateurs et ne comporte pas de défauts. Le fait qu'un logiciel compile ne suffit donc pas : il doit également être correct, fiable et performant.

Dans le cadre de **TicketPulse**, la stratégie de tests doit permettre de vérifier le fonctionnement des principales fonctionnalités de la plateforme, mais également de réduire les risques associés aux opérations critiques telles que la réservation simultanée d'un siège, le paiement et la génération du billet électronique.

Le plan de tests proposé combine plusieurs niveaux de tests afin de valider progressivement les composants individuels, leurs interactions et finalement le fonctionnement global de TicketPulse.

1. **Objectifs du plan de tests**

Le plan de tests de TicketPulse poursuit les objectifs suivants :

- vérifier que chaque fonctionnalité produit le résultat attendu ;
- détecter les défauts avant la mise en production ;
- vérifier les interactions entre les différents composants ;
- garantir qu'un siège ne puisse pas être vendu deux fois ;
- vérifier le processus complet de réservation et d'achat ;
- vérifier la génération du billet électronique et du QR Code ;
- vérifier l'envoi de la confirmation d'achat ;
- détecter les régressions après une modification du système ;
- vérifier que le système répond aux principales exigences définies pour TicketPulse.

2. **Stratégie générale de tests**

Il existe plusieurs niveaux et catégories de tests complémentaires. Pour TicketPulse, nous retenons les **tests unitaires, tests d'intégration, tests système, tests fonctionnels, tests d'acceptation et tests de régression**.

Cette combinaison permet de tester TicketPulse progressivement, depuis ses composants individuels jusqu'au système complet.

Type de test	Objectif dans TicketPulse	Exemples
Tests unitaires	Vérifier individuellement les fonctions et composants du système.	Vérifier le calcul du prix d'un billet, l'expiration d'une réservation ou la génération d'un QR Code.
Tests d'intégration	Vérifier la communication entre plusieurs composants.	Vérifier l'interaction entre réservation et paiement ou entre achat et génération du billet.
Tests système	Vérifier TicketPulse dans son ensemble dans un environnement proche de la production.	Simuler le parcours complet : recherche → sélection → réservation → paiement → billet.
Tests fonctionnels	Vérifier que les fonctionnalités répondent aux exigences définies.	Vérifier la recherche d'événements, la consultation des sièges et la réservation.
Tests d'acceptation	Vérifier que le système répond aux besoins et critères attendus avant sa livraison.	Confirmer qu'un utilisateur peut acheter correctement un billet et recevoir sa confirmation.
Tests de régression	Vérifier qu'une modification n'a pas détérioré une fonctionnalité existante.	Après une modification du module de paiement, vérifier que réservation, paiement et génération du billet fonctionnent toujours.

3. Tests unitaires

Les tests unitaires constituent la base de la stratégie de tests. Le professeur indique également qu'un bon test unitaire doit être **indépendant, reproductible, rapide, simple, lisible et automatisé**

Pour TicketPulse, ils pourront notamment concerner:

ID	Composant testé	Test	Résultat attendu
TU01	Réservation	Réserver un siège disponible.	Le siège passe à l'état temporairement réservé.
TU02	Réservation	Tenter de réserver un siège déjà réservé.	La nouvelle réservation est refusée.
TU03	Expiration	Atteindre la durée limite d'une réservation temporaire.	Le siège redevient disponible.
TU04	Billet	Générer un billet après confirmation de l'achat.	Un billet électronique est créé.
TU05	QR Code	Générer le QR Code associé au billet.	Un QR Code est associé au billet.
TU06	Paiement	Traiter une réponse de paiement acceptée.	L'achat est confirmé.
TU07	Paiement	Traiter une réponse de paiement refusée.	L'achat n'est pas confirmé.

4. Tests d'intégration

Les tests d'intégration serviront à vérifier que les composants de TicketPulse travaillent correctement ensemble.

ID	Composants concernés	Scénario	Résultat attendu
TI01	Réservation + paiement	L'utilisateur réserve un siège puis effectue son paiement.	La réservation est correctement associée au paiement.
TI02	Paiement + billet	Le paiement est accepté.	Le système déclenche la génération du billet électronique.
TI03	Billet + QR Code	Un billet électronique est généré.	Un QR Code est correctement associé au billet.
TI04	Achat + courriel	L'achat est confirmé.	Une confirmation d'achat est envoyée à l'utilisateur.
TI05	Réservation + disponibilité	Un utilisateur réserve un siège.	Le siège n'apparaît plus comme disponible pour un autre utilisateur.

5. Tests fonctionnels

Les tests fonctionnels sont basés sur les exigences fonctionnelles du cahier des charges et visent à s'assurer que chaque fonctionnalité produit le résultat attendu en réponse aux actions de l'utilisateur. Ils considèrent le logiciel comme une **Black Box**, sans nécessiter la connaissance du code.

ID	Fonctionnalité	Action testée	Résultat attendu
TF01	Recherche	Rechercher un événement existant.	L'événement correspondant est affiché.
TF02	Billets disponibles	Consulter les billets d'un événement.	Les billets disponibles sont affichés.
TF03	Plan de salle	Ouvrir le plan de salle.	Le plan et l'état des sièges sont affichés.
TF04	Réservation	Sélectionner un siège disponible.	Le siège est temporairement réservé.
TF05	Paiement	Payer une réservation valide.	Le paiement est traité et l'achat confirmé s'il est accepté.
TF06	Billet électronique	Finaliser un achat.	Un billet électronique avec QR Code est généré.
TF07	Confirmation	Finaliser l'achat.	Une confirmation est envoyée par courriel.
TF08	Concurrence	Deux utilisateurs tentent d'acheter le même siège.	Un seul utilisateur peut obtenir le siège.

Le **TF08 est particulièrement critique**, puisque l'énoncé impose explicitement que le même siège ne puisse jamais être vendu deux fois.

6. Tests système

Les tests système valident le fonctionnement complet du système dans un environnement proche de celui de production.

Pour TicketPulse, un scénario système représentatif serait :

Scénario TS01 — Achat complet d'un billet

1. L’utilisateur accède à TicketPulse.
2. Il recherche un événement.
3. Il consulte les billets disponibles.
4. Il ouvre le plan de salle.
5. Il sélectionne un siège disponible.
6. TicketPulse réserve temporairement le siège.
7. L’utilisateur effectue son paiement.
8. Le paiement est accepté.
9. TicketPulse confirme l’achat.
10. Un billet électronique contenant un QR Code est généré.
11. Une confirmation est envoyée par courriel.
12. Le siège n’est plus disponible à la vente.

Résultat attendu : l’ensemble du parcours doit se dérouler correctement sans incohérence entre les différents composants.

7. Scénarios d’acceptation

Les scénarios d’acceptation permettent de vérifier que les exigences essentielles de TicketPulse sont satisfaites du point de vue de l’utilisateur.

ID	Scénario	Précondition	Action	Critère d’acceptation
SA01	Recherche d’événement	Des événements existent.	L’utilisateur effectue une recherche.	Les événements correspondant aux critères sont affichés.
SA02	Réservation d’un siège	Le siège est disponible.	L’utilisateur sélectionne le siège.	Le siège devient temporairement réservé.
SA03	Double réservation	Deux utilisateurs ciblent le même siège.	Les deux tentent de le réserver.	Un seul obtient la réservation.
SA04	Paiement accepté	Une réservation valide existe.	Le paiement est accepté.	L’achat est confirmé.
SA05	Paiement refusé	Une réservation valide existe.	Le paiement est refusé.	Aucun achat n’est confirmé.
SA06	Génération du billet	Le paiement est confirmé.	Le système finalise l’achat.	Un billet électronique avec QR Code est généré.
SA07	Confirmation par courriel	L’achat est terminé.	Le système traite la confirmation.	Le courriel de confirmation est envoyé.

SA08	Expiration de réservation	Une réservation temporaire n'est pas finalisée.	La durée maximale est atteinte.	Le siège redevient disponible.
SA09	Durée du paiement	Le client commence son paiement.	Il réalise le processus de paiement.	Le paiement peut être finalisé en moins de 5 minutes.
SA10	Temps de réponse	Le système reçoit une requête utilisateur.	La requête est traitée.	La majorité des requêtes obtiennent une réponse en moins de 2 secondes.

Les deux derniers critères correspondent directement aux exigences quantitatives fournies dans l'énoncé de TicketPulse.

8. Automatisation et intégration continue

Une partie importante des tests doit être automatisée afin qu'ils puissent être exécutés régulièrement.

L'automatisation permet d'exécuter les tests plus rapidement, de réduire les erreurs humaines, de vérifier les régressions et de faciliter l'intégration continue. Dans un pipeline CI/CD, les tests peuvent être exécutés automatiquement à chaque modification et un échec peut bloquer le déploiement.

Pour TicketPulse, les tests unitaires et une partie des tests d'intégration et fonctionnels pourront donc être automatisés. Lorsqu'une modification est envoyée dans le dépôt Git, la suite de tests pourra être exécutée afin de détecter rapidement une éventuelle régression.

9. Conclusion

La stratégie de tests de TicketPulse repose sur plusieurs niveaux complémentaires permettant de vérifier progressivement le système, depuis les composants individuels jusqu'au parcours complet d'achat. Les tests unitaires et d'intégration permettent de détecter rapidement les défauts techniques, tandis que les tests fonctionnels, système et d'acceptation permettent de vérifier la conformité de TicketPulse aux exigences du projet.

Une attention particulière doit être accordée aux fonctions critiques, notamment la réservation temporaire, la prévention des doubles ventes, le paiement et la génération du billet électronique. L'automatisation des tests et leur intégration dans un pipeline CI/CD permettront également de détecter les régressions lors des évolutions futures du système.